

Обзор литературы: автоматический перевод локализационных файлов

1 Введение

Задача автоматического перевода локализационных файлов (в нашем случае — файлов игровых модификаций) является комплексной и может быть разделена на несколько подзадач:

1. Получение текста для перевода из локализационного файла;
2. Перевод текста;
3. Сохранение разметки, содержащейся в исходном тексте;
4. Обеспечение терминологической консистентности;
5. Оценка качества перевода.

Литература, относящаяся к первой задаче, рассматриваться не будет, поскольку она касается технической реализации конечного продукта и зависит от структуры и формата конкретного локализационного файла.

Четвёртая задача не входит в рамки MVP, однако представляет значительный интерес и потенциал для дальнейшего развития продукта.

В контексте данного обзора в приоритете рассматривались работы, ориентированные на перевод текста, содержащий в себе разметку какого-либо вида, однако переход к рассмотрению статей такого рода стоит начинать с изучения основ, а именно с рассмотрения материала, относящегося к первой задаче.

2 Состояние машинного перевода

Автоматический машинный перевод пережил стремительный рост качества за последнее десятилетие благодаря переходу от статистических методов к нейронным моделям [3]. Появление больших языковых моделей (LLM) расширило возможности перевода, позволяя одной модели выполнять перевод между множеством языков наряду с другими задачами обработки текста [5, 18].

За последние годы нейронный машинный перевод стал де-факто стандартом, вытеснив статистические методы благодаря способности нейросетевых архитектур моделировать перевод от исходной строки к целевой [3] (возможность перевода не по словам/фразам, а по связкам текста). Крупные технологические компании (Google, Microsoft, Amazon и др.) внедрили нейронный машинный перевод (NMT) в свои продукты уже к 2016–2018 гг., что свидетельствует о промышленной зрелости технологий перевода [3]. Нейронные модели, особенно основанные на архитектуре *Трансформер*, достигли высокого качества перевода на многих языковых парах [3].

Почему именно *Трансформер*? Архитектура *Трансформер* стала стандартом в НМП благодаря механизму самовнимания (*self-attention*), который позволяет каждой позиции учитывать весь контекст и точно моделировать дальние зависимости между исходным и переводом [3]. Рост качества при увеличении размера модели, сквозное внимание и совместимость *Трансформер* с многоязычным предобучением (например, mBART) позволили существенно повысить качество перевода, особенно для малоресурсных языков ¹ [19].

В то же время появление LLM дало новый импульс развитию МП. LLM, обученные на огромных массивах текстов, способны выполнять перевод без явного специализированного обучения на параллельных корпусах, используя обучение на примерах в промпте (*in-context learning*) [5, 18]. Исследования показывают, что GPT-4 уже может соперничать с лучшими специализированными системами перевода: в оценке на множестве языковых направлений GPT-4 превзошёл сильную многоязычную модель NLLB примерно в 40% языковых пар; однако даже такие LLM пока уступают коммерческим системам на многих направлениях, особенно для языков с малым количеством данных [18].

3 Перевод с сохранением разметки

Перевод текстов с сохранённой разметкой (т. е. с HTML, XML или другими встроенными тегами) представляет собой сложную задачу, поскольку помимо языкового соответствия требуется обеспечить корректную передачу всех тегов и их структуры в тексте перевода. Традиционным решением долгое время был подход с удалением тегов и последующей проекцией (*detag-and-project*), при котором входной текст очищается от тегов перед переводом, а затем удалённые теги проецируются обратно в переведённый текст на основе выравнивания слов между исходным и целевым предложениями. Этот метод подвержен накоплению ошибок: неточности машинного перевода, ошибки выравнивания и проекции могут привести к потере тегов или их неправильному расположению. [1].

Альтернативой служат методы, интегрирующие теги непосредственно в процесс перевода. Один из них — включение тегов в обучающие данные в виде специальных токенов, где, например, символы `<tag>` и `</tag>` рассматриваются как элементы текста. Модель обучается воспроизводить эти токены в выходной последовательности, тем самым формируя перевод с сохранённой структурой разметки. Вариант

¹По сути текст с разметкой можно представить в виде малоресурсного языка и перенести на него все подходы что применимы к ним (если обучать модельку свою)

данного подхода — техника *маскирования* (*placeholdering*), при которой содержимое тегов или сами теги временно заменяются на специальные маркеры, а после перевода выполняется обратная замена. Этот метод подробно исследован Post и соавт.² [2]: авторы показали, что маскирование помогает избежать потерь специфических токенов (например, имён собственных, чисел, URL-адресов), но может снижать качество перевода окружающего текста.

Hanneman и Dinu [1] предложили другой подход — включение тегов непосредственно в процесс обучения модели. Они исследовали несколько стратегий: (1) удаление тегов с последующей проекцией (*detag-and-project*); (2) маскирование тегов абстрактными метками (например, `<x1>`–`</x1>`); (3) расширение обучающих данных путём *вставки синтетических тегов* (*tag injection*).³ Последний метод позволил увеличить устойчивость модели к структурным вариациям разметки. Авторы показали, что при достаточном объёме синтетически размеченных данных ⁴ точность переноса тегов достигает 95–99%.

Отдельное направление исследований рассматривает сохранение разметки как задачу постобработки. Так, Zenkel и коллеги [14] разработали подход, в котором задача переноса тегов решается независимо от основного перевода: с помощью специализированного модуля теги вставляются обратно в уже готовый перевод в соответствии с выравниванием между исходным и целевым предложениями. Аналогичные идеи реализованы в системе *TransIns*, где теги восстанавливаются после перевода с использованием выравнивания слов и набора правил [15]. Такие методы позволяют обеспечить структурную целостность и избежать ошибок вложенности, однако их качество напрямую зависит от точности выравнивания между исходным и переведённым текстом и от степени различия в синтаксисе языковой пары.⁵

С развитием больших языковых моделей (LLM) появилась возможность перевода размеченных текстов без изменения архитектуры модели. Dabre и др. [17] исследовали, как многоязычные LLM (в частности, BLOOM и BLOOMZ) справляются с задачей переноса тегов при использовании подсказок (*prompting*). Авторы показали, что при *few-shot* обучении LLM способны правильно копировать и размещать теги, зачастую превосходя классические системы *detag-and-project* по точности позиционирования тегов. В последующих работах Dabre и коллег (2024) проведены эксперименты с дообучением LLM на синтетических корпусах с тегами, созданных методом автоматической вставки разметки, аналогичным подходу Hanneman и Dinu [1]. Это позволило повысить стабильность генерации и улучшить сохранение тегов без значительного ухудшения качества перевода.⁶

²я и дальше буду использовать такой формат для указания исследования, т.к. это супер удобно и не надо придумывать фразы вроде: "в исследовании посвященном тому-то было то-то"

³Вот здесь исследователи использовали подход с восстановлением тегов по правилам (*project*), но он у них не завелся нормально и они сказали что лучше дотренировать маски и получить топ вариант, чем мучить правила. С чем я согласен, применять правила для перевода с иероглифов (китайский, корейский, японский) гиблое дело.

⁴Для создания таких датасетов вполне нормально применяют аугментацию, например у Ryu и др. [16] был генератор, но на HTML-разметку.

⁵Как уже говорил - вероятно не пойдет. Хотя я думал про реализацию этого модуля в виде отдельно натренированной модельки, которая будет получать исходник с тегами, перевод без тегов, и в итоге ставить теги. Но сложность с иероглифами вероятно будет очень большой. Но датасет не слишком сложно делать - просто удалить теги из имеющихся языковых пар. Можно пробовать ради эксперимента, тем более статей про такое я не видел

⁶Как будто *good to go* вариант, но для оправданного уровня качества перевода надо брать модель

4 Обучение и подготовка моделей

Для эффективного перевода текстов со структурной разметкой (HTML, XLIFF и др.) применяются методы дообучения и адаптации существующих моделей, без обучения «с нуля».

Доменное дообучение моделей. Один из базовых методов — дообучение нейронной МТ-модели на параллельных данных из нужной предметной области. Модель, предварительно обученная на общих данных, дообучается на корпусе целевого домена, что повышает качество перевода и соблюдение терминологии и стиля домена [3]. На практике специализированная точная настройка (fine-tuning) заметно улучшает качество по сравнению с общедоменной моделью [3]. Open-source фреймворки (Marian NMT, OpenNMT) позволяют обучать кастомные модели или дообучать предобученные под конкретный домен; при обслуживании нескольких доменов полезны параметро-эффективные методы (адаптеры, LoRA), при которых базовая модель остаётся общей, а малые модули обучаются под каждый домен.⁷

Дистилляция знаний и стратегия *distill+adapt*. Для повышения качества на целевом домене при ограниченных ресурсах применяют дистилляцию знаний — перенос знаний от крупной «учителя»-модели к более компактной «студенту»-модели. Последовательная схема «distill, затем adapt» показала высокую эффективность: сначала сжатие общей модели дистилляцией, затем адаптация под домен и повторная дистилляция дают маленьким моделям высокое качество при малом числе параметров [4]. На практике «учителем» может выступать как открытая NMT/мультязычная модель, так и LLM через API (например, для генерации псевдопереводов обучающей выборки).⁸

4.1 Перевод с сохранением терминологии

Сохранение терминологии является ключевой задачей в прикладных системах машинного перевода. Среди классических подходов одним из первых предложено **ограниченное декодирование (constrained decoding)**, при котором модель принудительно включает заранее указанные термины в выходной текст. Эта техника часто реализуется через модифицированные алгоритмы лучевого поиска (beam search) с использованием конечных автоматов [2, 1]. Ограниченное декодирование гарантирует включение терминов, но существенно увеличивает вычислительную сложность и может вызывать ошибки (дублирование, неправильный порядок слов) при большом количестве ограничений [2].⁹

Другой распространённый подход основан на **маркировании и подстановке терминов**: методы «placeholder» и «code-switch» [2, 14]. В методе placeholder исходные термины заменяются на условные метки (например, «T1», «T2»), и модель обучают

которую не развернуть на локальном компе

⁷Вариант с лорой под разметку выглядит очень заманчиво

⁸Это конечно все про малоресурсные языки, и проблема в тут вероятно будет в том что наш язык супер похож (тот же язык но с разметкой) и внутренности модели просто не вывезут разницу

⁹И при этом надо разметку сохранять - варик не совсем рабочий

предсказывать эти метки. Это обеспечивает высокую точность включения терминов, но утрачивается семантика «пустышек»¹⁰. В методе code-switch вместо меток просто вставляются готовые переводы терминов в исходный текст. Zhang и соавт. (2023) предложили гибрид PH+CS: на этапе обучения модель учит предсказывать метки, а при выводе эти метки заменяются на целевые термины.

LLM легко воспринимают инструкции, что позволяет в явном виде включать термины в подсказку. Например, Huang и соавт. (2024) предложили схему *Translate-and-Revise (TAR)*: на первом шаге LLM переводит с учётом указаний, а затем на втором шаге через уточняющий промпт исправляет пропущенные термины [6]. Другой класс методов базируется на *генерации, дополненной поиском (RAG)*: в контекст LLM добавляется внешняя информация (словари, базы терминов). Chen и др. (2024) описали многошаговую цепочку с RAG: сначала из исходного текста извлекаются ключевые термины, затем их переводы из двуязычного словаря встраиваются в контекст через RAG¹¹, после чего LLM итеративно уточняет перевод [7].

5 Метрики проверки качества перевода

Для оценки качества автоматического перевода используются как классические лингвистические метрики, так и специализированные метрики для проверки корректности разметки (markup). В таблице 2 представлены основные из них.¹²

¹⁰Вот этот подход вот да, очень соблазнителен

¹¹Вариант вероятно рабочий - можно еще и по ходу перевода заполнять векторную базу примерами перевода с новыми терминами, и потом по ним сверять. Правда надо будет детектить термин - записывать в векторную - либо искать похожее и корректировать перевод

¹²Метрики* выведены мной из совокупности похожих для конкретных форматов тегов (XML/HTML)

Метрика	Описание	Особенности / ограничения
BLEU	Оценивает перевод через совпадение n-грамм (последовательностей из n слов) между переводом-кандидатом и эталонными переводами. Простая и быстрая в вычислении.	Не учитывает семантику. Сильно наказывается использование синонимов или перефразирование по сравнению с эталоном.
METEOR	Учитывает не только точное совпадение слов, но и их основы (стеммы), а также синонимы и порядок слов. Строит выравнивание кандидата и эталона с учётом лексических преобразований.	Лучше коррелирует с человеческими оценками, чем BLEU, благодаря учёту семантических соответствий. Однако требует лексических ресурсов (словари синонимов, WordNet и т.д.) для сопоставления слов.
TER (Translation Edit Rate)	Измеряет редакционное расстояние: минимальное число операций вставки, удаления, замены, перестановки слов, необходимых для превращения в эталон.	Оценивает объём пост-редактирования (хорошо отражает усилия переводчика), но не анализирует адекватность смысла. Более низкий TER означает лучший перевод.
chrF / chrF++	F-метрика совпадения символьных n-грамм (обычно до 6-грамм). Считает F1 по пересечению символов (точность + полноту) между переводом и эталоном.	Особенно эффективна для морфологически богатых языков: менее чувствительна к свободному порядку слов и неточностям сегментации. Однако игнорирует грамматическую структуру и смысл; учитывает лишь поверхностное совпадение символов.

Таблица 2: Метрики оценки качества перевода.

Метрика	Описание	Особенности / ограничения
COMET	Нейросетевая метрика на основе глубоких моделей (BERT и др.). Обучается на больших наборах с оценками качества перевода экспертами и выдаёт предсказание корреляции с человеческой оценкой.	Хорошо учитывает адекватность и плавность перевода, показана высокая корреляция с оценками людей.
NIST	Версия BLEU от NIST, взвешивающая n-граммы по их информационной ценности. Редкие (информативные) n-граммы приносят больший вклад в оценку, чем часто встречающиеся.	Менее чувствителен к изменениям длины перевода. Как и BLEU, не оценивает семантику. Диапазон значений менее нагляден, чем у BLEU; редко используется отдельно.
BERTScore	Метрика на основе контекстных эмбедингов. Для каждого токена в переводе и эталоне вычисляется косинусное сходство эмбедингов, затем агрегируется в F1.	Учитывает семантическое сходство (синонимы, перефразировки), даёт более высокую корреляцию с человеческой оценкой, чем n-граммовые метрики.
BLEURT	Обучаемая метрика: трансформер предварительно обучается на больших данных, затем дообучается на парах перевод–оценка.	Хорошо коррелирует с человеческими оценками (учитывает перефразирование и контекст).
Compilability*	Метрика корректности разметки: доля переводов, образующих валидный скелет (все теги правильно закрыты и вложены).	Проверяет только синтаксис: если хотя бы один тег отсутствует или неправильно вложен, перевод считается «невалидным».
Tag Match*	Метрика структуры: доля переводов, у которых структура точно совпадает со структурой эталона (теги стоят в тех же местах и с теми же атрибутами).	Очень строгая метрика «всё или ничего»: даже небольшое отличие в порядке/глубине тегов обнуляет балл.
Tag F1 (F₁ по тегам)	Метрика точности разметки: вычисляет F1-меру для лексем, охваченных тегами. Считается перекрытие (precision/recall) областей текста, обрамлённых тегами в переводе и эталоне.	Оценивает точность размещения тегов: штрафует за пропуски и неточности в охвате слов. Однако игнорирует порядок тегов и их вложенность.
Flagrant Tag Errors	Метрика грубых ошибок переноса разметки: подсчитывает количество «флагрантных» ошибок — упущенных, добавленных, искажённых или неправильно вложенных тегов.	Выявляет только явные ошибки в разметке.

6 Заключение

Анализ существующих подходов к сохранению разметки и терминологической целостности при машинном переводе показывает, что наилучшие результаты достигаются либо при использовании синтетически размеченных корпусов и дообучении моделей на специализированных данных, либо при использовании LLM с системами для проверки результатов. Методы, основанные на правилах или жёстких ограничениях при декодировании, демонстрируют низкую устойчивость и плохо масштабируются, особенно для языков с иероглифическим письмом. При движении в сторону создания своей модели оптимальной стратегией представляется применение параметро-эффективных дообучаемых архитектур и использование техник маскирования, «placeholder» и «code-switch». Перспективным направлением является интеграция внешних терминологических ресурсов через RAG.

По сути:

1. Использование LLM-моделей оправдано по качеству перевода¹³, по возможности сохранения разметки и по возможности сохранения терминологии¹⁴.
2. Возможно экспериментировать с простыми LLM для использования их в качестве `detag-and-project` модуля¹⁵.
3. Маскирование для LLM или другой системы перевода также является вариантом для рассмотрения.
4. Следует оценить состояние локализационных файлов в датасете на предмет параллельности переводов, плотности и глубины тегов в разметке¹⁶.
5. Можно начать поиск моделей для дообучения на сохранение разметки.

¹³Нет малоресурсных языков

¹⁴При помощи дополнительных промптов или RAG

¹⁵Не видел этого в статьях, но судя по общей картине на готовом переводе и с исходником задача должна быть выполнима.

¹⁶Это поможет и при аугментации данных, и для общего понимания сложности перевода.

Список литературы

- [1] Hanneman G., Dinu G. (2020). *How Should Markup Tags Be Translated?* Proceedings of the 5th Conference on Machine Translation (WMT 2020), pp. 1160–1173.
- [2] Post M., Ding S., Martindale M. J., Wu W. (2019). *An Exploration of Placeholder in Neural Machine Translation*. MT Summit XVII, Vol.1, pp. 182–192.
- [3] Stahlberg F. (2020). *Neural Machine Translation: A Review*. Journal of Artificial Intelligence Research, 69:343–418.
- [4] Gordon M. A., Duh K. (2020). *Distill, Adapt, Distill: Training Small, In-Domain Models for NMT*. Proceedings of the 4th Workshop on Neural Generation and Translation, pp. 110–118.
- [5] Wang Y. et al. (2024). *Recent Advances in Interactive Machine Translation With Large Language Models*. IEEE Access, 12:179353–179366.
- [6] Huang P. et al. (2025). *Translate-and-Revise: Boosting LLMs for Constrained Translation*. Chinese Computational Linguistics (CCL 2024), LNCS 14761, pp. 120–139.
- [7] Arslan M. et al. (2024). *A Survey on Retrieval-Augmented Generation with LLMs*. Procedia Computer Science, 246:3781–3790.
- [8] NLLB Team et al. (2022). *No Language Left Behind: Scaling Human-Centered Machine Translation*. arXiv preprint arXiv:2207.04672.
- [9] Seamless Communication et al. (2023). *SeamlessM4T: Massively Multilingual and Multimodal Machine Translation*. arXiv preprint arXiv:2308.11596.
- [10] Seamless Communication et al. (2023). *SeamlessExpressive: Multilingual Expressive and Streaming Speech Translation*. arXiv preprint arXiv:2312.05187.
- [11] Kocmi T. et al. (2024). *The LLM Era Is Here but MT Is Not Solved Yet: Findings of the WMT24 General MT Task*. Proceedings of the Seventh Conference on Machine Translation (WMT 2024).
- [12] Rei R. et al. (2023). *Scaling up COMETKiwi: Unbabel-IST 2023 Submission for Quality Estimation*. Proceedings of WMT 2023.
- [13] Google Research (2022). *WMT MQM Human Evaluation Repository*. GitHub, 2020–2023.
- [14] Zenkel T., Wuebker J., DeNero J. (2021). *Tag Insertion: Automated Post-Editing for Markup Transfer*. Proceedings of the 16th Conference of the European Chapter of the ACL, pp. 1851–1863.
- [15] Steffen J., Van Genabith J. (2021). *TransIns: Document Translation with Markup Reinsertion*. Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 28–34.

- [16] Ryu S., Choi Y., Lee J. (2022). *Efficient Synthetic Data Generation for Markup-Aware Neural Machine Translation*. Proceedings of the 19th International Workshop on Spoken Language Translation (IWSLT 2022), pp. 412–420.
- [17] Dabre R., Tanaka H., Buschbeck B., Exel M. (2023). *A Study on the Effectiveness of Large Language Models for Translation with Markup*. Proceedings of Machine Translation Summit XIX, Vol. 1, pp. 148–159.
- [18] Ataman, D., Birch, A., Habash, N., Federico, M., Koehn, P., & Cho, K. (2025). *Machine Translation in the Era of Large Language Models: A Survey of Historical and Emerging Problems*. Information, 16(9), 723.
DOI: 10.3390/info16090723.
- [19] Liu, Y., et al. (2020). *Multilingual Denoising Pre-training for Neural Machine Translation (mBART)*. Transactions of the Association for Computational Linguistics, 8, 726–742.
DOI: 10.1162/tacl_a_00343.